CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment – 4

Name: Aariz Raza
UID: 24BAI71000
Branch: CSE AIML (CS221)
Section/Group: 24AIT_NTPP-3[B]
Semester: 5th
Date of Performance: 30/07/2026
Subject: Full Stack-II
Subject Code: 24CSP-337

EXPERIMENT – 4.1

1. **AIM:** To design and implement an interactive calendar interface for scheduling and managing posts.
2. **OBJECTIVE:**

- To understand time-based data visualization in UI systems.
- To implement calendar-based scheduling interfaces.
- To map structured data to temporal layouts.
- To enable user interactions such as drag-and-drop.

3. **SOFTWARE REQUIREMENTS:**

- React.js
- FullCalendar (or any Calendar Library)
- Node.js
- npm (Node Package Manager)
- Visual Studio Code
- Google Chrome / Microsoft Edge
- Date Utility Libraries (Optional)

4. **THEORY**

An interactive calendar is an important user interface component used to display and manage time-based information in an organized manner. It is commonly used in applications such as social media schedulers, event management systems, appointment booking platforms, and project management tools. A calendar allows users to schedule, edit, and manage events by assigning them to specific dates and time slots.

In this experiment, an interactive calendar is developed using React.js and a calendar library such as FullCalendar. Posts are mapped to different dates and displayed as calendar events. The application dynamically renders scheduled posts and allows users to interact with them by clicking, editing, or rescheduling events. Advanced features such as drag-and-drop enable users to move events from one date to another without manually updating the schedule. The calendar is synchronized with the application's state so that any modification is reflected immediately on the user interface. This experiment introduces concepts such as time-based data visualization, event mapping, state synchronization, dynamic rendering, user interaction, and drag-and-drop functionality, which are essential for developing modern scheduling and planning applications.

5. IMPLEMENTATION/PROCEDURE

- Create a React project.
- Install a calendar library (e.g., FullCalendar).
- Design the calendar interface (Day, Week, or Month view).
- Create sample post/event data.
- Map post data to calendar events.
- Display events dynamically on the calendar.
- Implement click events to view or edit posts.
- Enable drag-and-drop for rescheduling events.
- Synchronize the calendar with application state.
- Test scheduling and event management features.

6. CODE:

```
import FullCalendar from "@fullcalendar/react";
import dayGridPlugin from "@fullcalendar/daygrid";

function App() {
 const events = [
  {
   title: "Instagram Post",
   date: "2026-08-10",
  },
  {
   title: "LinkedIn Post",
   date: "2026-08-12",
  },
 ];

 return (
  <div>
   <h2>Post Scheduling Calendar</h2>

   <FullCalendar
    plugins={[dayGridPlugin]}
    initialView="dayGridMonth"
    events={events}
   />
  </div>
 );
}

export default App;
```

7. OUTPUT

The application successfully displays an interactive calendar where scheduled posts appear on their assigned dates. Users can view, manage, and modify scheduled events through the calendar interface. The calendar provides an organized view of posts, making content planning and scheduling easier.

Fig 4.1: Interactive Calendar for Post Scheduling

8. LEARNING OUTCOME

- Understand time-based data visualization.
- Learn to implement an interactive calendar interface.
- Map structured data to calendar events.
- Display events dynamically using React.
- Handle user interactions such as click and drag-and-drop.
- Synchronize calendar events with application state.
- Improve scheduling and content management techniques.
- Develop user-friendly scheduling applications using React.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 4.2

1. AIM:
To optimize rendering performance and implement testing strategies for interactive UI components.

2. OBJECTIVE:

- To understand performance bottlenecks in UI systems.
- To optimize rendering using memoization techniques.
- To reduce unnecessary re-renders.
- To implement testing for UI components and logic.

3. SOFTWARE REQUIREMENTS:

- React.js
- Jest
- React Testing Library
- Browser DevTools
- Visual Studio Code
- Node.js
- npm (Node Package Manager)

4. THEORY

Modern React applications often contain interactive components that update frequently based on user actions. If these components re-render unnecessarily, application performance decreases, resulting in slower response times and poor user experience. Performance optimization aims to reduce unnecessary rendering and improve the efficiency of React applications.

In this experiment, React provides several optimization techniques such as React.memo(), useMemo(), and useCallback(). The React.memo() function prevents a component from re-rendering unless its props change. The useMemo() Hook stores the result of expensive calculations and recalculates them only when required, reducing computation time. The useCallback() Hook memorizes functions so that they are not recreated during every render, helping child components avoid unnecessary updates.

The experiment also introduces software testing using Jest and React Testing Library. Testing ensures that user interface components work correctly under different scenarios, including user interactions such as clicking buttons, updating data, and rendering components. Combining performance optimization with testing results in applications that are faster, reliable, scalable, and easier to maintain.

5. IMPLEMENTATION/PROCEDURE

1. Create a React application.
2. Identify components that re-render frequently.
3. Apply React.memo() to child components.
4. Use useMemo() for expensive calculations.
5. Use useCallback() for event handler functions.
6. Optimize state updates.
7. Write unit tests using Jest.
8. Test UI interactions using React Testing Library.
9. Verify optimized rendering behavior.
10. Execute tests and validate application performance.

6. CODE

```
import React, { useState, useMemo, useCallback } from "react";

// Memoized Child Component
const StudentList = React.memo(({ students }) => {
  console.log("Student List Rendered");

  return (
    <div
      style={{
        marginTop: "25px",
        background: "#ffffff",
        borderRadius: "12px",
        padding: "20px",
        boxShadow: "0 6px 15px rgba(0,0,0,0.1)",
      }}
    >
      <h2 style={{ color: "#2c3e50", marginBottom: "15px" }}>
        Student List
      </h2>

      {students.map((student, index) => (
        <div
          key={index}
          style={{
            padding: "10px",
            marginBottom: "10px",
            borderRadius: "8px",
            background: "#f4f6f8",
            fontWeight: "500",
            color: "#34495e",
          }}
        >
          {student}
        </div>
      ))}
    </div>
  );
});

function App() {
  const [count, setCount] = useState(0);
```

```
// Memoized Student Data
const students = useMemo(() => {
  return [
    "Aariz",
    "Aariz_2",
    "Aariz_3",
    "Aariz_4",
    "Aariz_5",
  ];
}, []);

// Memoized Function
const increaseCounter = useCallback(() => {
  setCount((prev) => prev + 1);
}, []);

return (
  <div
    style={{
      minHeight: "100vh",
      background: "linear-gradient(to right, #4facfe, #00f2fe)",
      display: "flex",
      justifyContent: "center",
      alignItems: "center",
      fontFamily: "Arial, sans-serif",
    }}
  >
    <div
      style={{
        width: "450px",
        background: "#ffffff",
        padding: "30px",
        borderRadius: "15px",
        boxShadow: "0 10px 25px rgba(0,0,0,0.2)",
      }}
    >
      <h1
        style={{
          textAlign: "center",
          color: "#2c3e50",
          marginBottom: "20px",
        }}
      >
```

```
  ⚡ Performance Optimization
</h1>

<div
  style={{
    textAlign: "center",
    background: "#eef5ff",
    padding: "20px",
    borderRadius: "12px",
  }}
>
  <h2 style={{ color: "#2980b9", margin: "0" }}>
    Counter
  </h2>

  <h1
    style={{
      color: "#27ae60",
      fontSize: "45px",
      margin: "15px 0",
    }}
  >
    {count}
  </h1>

  <button
    onClick={increaseCounter}
    style={{
      background: "#3498db",
      color: "white",
      border: "none",
      padding: "12px 25px",
      borderRadius: "8px",
      fontSize: "16px",
      cursor: "pointer",
      transition: "0.3s",
    }}
    onMouseOver={(e) =>
      (e.target.style.background = "#2980b9")
    }
    onMouseOut={(e) =>
      (e.target.style.background = "#3498db")
    }
  >
```

```
          + Increase Counter
        </button>
      </div>

      <StudentList students={students} />
    </div>
  </div>
 );
}

export default App;
```

7. OUTPUT (SCREENSHOT OF WEB + explain)

The application displays a counter and a student list. Clicking the Increase Counter button updates only the counter, while the student list does not re-render because it is optimized using React.memo() and useMemo(). Unit testing verifies that the counter updates correctly, ensuring both performance optimization and functional correctness.

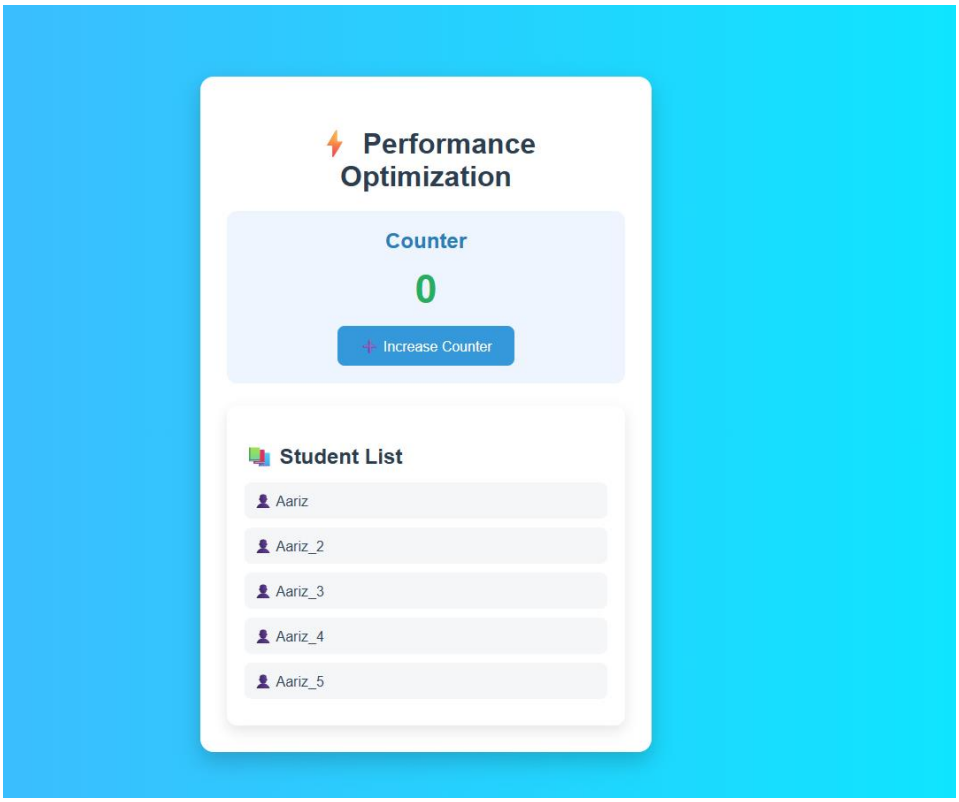

Fig 4.2: Performance Optimization and Testing of Interactive UI Components

8. LEARNING OUTCOME

- Understand performance bottlenecks in React applications.
- Learn to optimize rendering using React.memo().
- Implement useMemo() for expensive computations.
- Use useCallback() to optimize event handlers.
- Reduce unnecessary component re-rendering.
- Write unit tests using Jest.
- Test UI interactions using React Testing Library.
- Develop efficient, reliable, and scalable React applications.